



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Projekt i implementacja aplikacji webowej do transkrypcji nagrań
fortepianowych na nuty w formacie PDF z wykorzystaniem
zaawansowanych modeli AI

Michał JAGODA

Nr albumu: 300405

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr hab. inż. Karolina Nurzyńska

KATEDRA Algorytmiki i Oprogramowania

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2024

Tytuł pracy

Projekt i implementacja aplikacji webowej do transkrypcji nagrań fortepianowych na nuty w formacie PDF z wykorzystaniem zaawansowanych modeli AI

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

Design and implementation of a web application for transcribing piano recordings into musical notes in PDF format using advanced AI models

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Spis treści

1	Wstęp	1
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Osadzenie tematu w kontekście aktualnego stanu wiedzy	4
2.3	Studia literaturowe	5
3	Wymagania i narzędzia	9
3.1	Komponenty, moduły i biblioteki	9
3.2	Wymagania funkcjonalne	11
3.3	Wymagania niefunkcjonalne	12
3.4	Wymagania sprzętowe i programowe	13
3.4.1	Wymagania dla użytkowników końcowych	13
3.4.2	Wymagania dla środowiska serwerowego	14
3.5	Przypadki użycia	14
4	Specyfikacja zewnętrzna	15
4.1	Wymagania sprzętowe i programowe	15
4.1.1	Wymagania programowe	15
4.2	Kategorie użytkowników	15
4.3	Sposób obsługi	16
4.4	Administracja systemem	17
4.5	Kwestie bezpieczeństwa	17
4.6	Przykład działania	17
5	Specyfikacja wewnętrzna	19
5.1	Przedstawienie idei	19
5.2	Architektura systemu	19
5.3	Opis struktur danych i organizacji bazy danych	21
5.4	Skrypty deweloperskie	21
5.5	Przegląd ważniejszych algorytmów	22
5.6	Szczegóły implementacji wybranych fragmentów	23

5.6.1	Implementacja modelu transkrypcji	23
5.6.2	Implementacja systemu kolejkowania transkrypcji	24
5.6.3	Implementacja systemu SSE do monitorowania postępu	26
5.7	Diagramy UML	27
6	Model transkrypcji	33
6.1	Architektura modelu	33
6.2	Proces uczenia	34
6.2.1	Przetwarzanie wstępne danych	34
6.2.2	Parametry treningu	35
6.2.3	Funkcja straty	35
6.3	Wnioskowanie	35
7	Weryfikacja i walidacja	37
7.1	Organizacja testów	37
7.1.1	Testy jednostkowe (Unit Tests)	37
7.1.2	Testy integracyjne	37
7.2	Przypadki testowe i wykryte błędy	38
7.2.1	Testy automatyczne	38
7.2.2	Testy manualne	41
7.3	Testowanie wydajności	42
7.4	Najważniejsze pokrycie testami	42
8	Podsumowanie i wnioski	43
	Bibliografia	45
	Spis skrótów i symboli	49
	Źródła	51
	Lista dodatkowych plików, uzupełniających tekst pracy	53
	Spis rysunków	55
	Spis tabel	57

Rozdział 1

Wstęp

W dobie cyfrowej transformacji muzyki, potrzeba efektywnych narzędzi do konwersji nagrań audio na zapis nutowy staje się coraz bardziej istotna. Niniejsza praca przedstawia projekt i implementację systemu TuneScript, który wykorzystuje zaawansowane modele sztucznej inteligencji do automatycznej transkrypcji nagrań fortepianowych. System ten stanowi odpowiedź na rosnące zapotrzebowanie muzyków, kompozytorów i pedagogów muzycznych na narzędzia umożliwiające szybką i dokładną konwersję nagrań audio do formatu partytur.

Problem automatycznej transkrypcji muzyki fortepianowej jest szczególnie złożony ze względu na polifoniczny charakter instrumentu oraz bogactwo ekspresji muzycznej. Dotychczasowe rozwiązania często nie spełniają oczekiwań użytkowników w zakresie dokładności transkrypcji lub wygody użytkowania. TuneScript adresuje te wyzwania poprzez zastosowanie najnowszych osiągnięć w dziedzinie uczenia maszynowego i przetwarzania sygnałów audio.

Głównym celem pracy jest stworzenie kompleksowego systemu, który nie tylko dokładnie rozpoznaje nuty i ich parametry czasowe, ale także generuje profesjonalnie wyglądające partytury w formacie PDF. Cele szczegółowe obejmują:

- Implementację modelu uczenia głębokiego do precyzyjnej detekcji nut i ich parametrów.
- Opracowanie systemu konwersji plików MIDI do formatu PDF.
- Stworzenie intuicyjnego interfejsu webowego dla użytkowników.
- Zapewnienie mechanizmów zarządzania i udostępniania transkrypcji.

Zakres pracy obejmuje następujące obszary:

- Analiza istniejących rozwiązań i badań w dziedzinie automatycznej transkrypcji muzyki.

- Implementacja modelu uczenia maszynowego do transkrypcji.
- Projekt i implementacja systemu webowego.
- Integracja modelu uczenia maszynowego.
- Integracja mechanizmów generowania partytur.
- Testowanie i walidacja systemu.

Praca składa się z sześciu rozdziałów. Po wstępie, w rozdziale drugim przedstawiono szczegółową analizę problemu oraz przegląd literatury. Rozdział trzeci opisuje wymagania funkcjonalne i нефункционалне oraz wykorzystane narzędzia. Rozdział czwarty zawiera specyfikację zewnętrzną systemu, a rozdział piąty - specyfikację wewnętrzną wraz z opisem architektury i implementacji. Rozdział szósty przedstawia proces weryfikacji i walidacji systemu.

Rozdział 2

Analiza tematu

W dzisiejszym świecie cyfrowa muzyka fortepianowa jest szeroko rozpowszechniona, a dostępność nagrań w formatach takich jak MP3 znacznie ułatwia ich odtwarzanie. Jednocześnie powstaje potrzeba narzędzi, które umożliwią analizę i edycję muzyki w sposób bardziej zaawansowany, szczególnie dla muzyków, nauczycieli i entuzjastów fortepianu. Niniejsza praca podejmuje próbę stworzenia kompleksowego rozwiązania, które przekształca nagrania audio w czytelne partytury, wspierając edukację, kompozycję i praktykę muzyczną. Poniżej zaprezentowane są szczegółowe wyniki testów systemu, które ilustrują jego wydajność i precyzję działania w różnych scenariuszach.

2.1 Sformułowanie problemu

Współczesna muzyka fortepianowa często występuje w formie cyfrowych nagrań audio, najczęściej w formacie MP3. Format ten, choć wygodny do odtwarzania, nie pozwala na łatwą edycję czy analizę struktury muzycznej. Muzycy, kompozytorzy, pedagodzy muzyczni oraz entuzjaści fortepianu potrzebują narzędzia, które umożliwi im konwersję plików audio do formatu MIDI, a następnie do czytelnych partytur w formacie PDF.

Problem, którym zajmuje się niniejsza praca, polega na stworzeniu efektywnego, dokładnego i przyjaznego dla użytkownika systemu, który automatyzuje proces transkrypcji muzyki fortepianowej. System ten, nazwany TuneScript, ma za zadanie przekształcać nagrania fortepianowe z formatu MP3 do symbolicznej reprezentacji MIDI, a następnie generować profesjonalnie wyglądające partytury w formacie PDF.

Kluczowe wyzwania, które należy rozwiązać w ramach tego projektu, obejmują:

- Precyzyjne rozpoznawanie wysokości dźwięków w nagraniach fortepianowych, z uwzględnieniem złożoności polifonii fortepianowej.
- Dokładne określanie czasu rozpoczęcia i zakończenia każdej nuty, co jest kluczowe dla zachowania rytmu i tempa utworu.

- Odzworowanie dynamiki gry, która jest istotnym elementem ekspresji muzycznej na fortepianie.
- Efektywne przetwarzanie długich utworów muzycznych przy zachowaniu wysokiej jakości transkrypcji.
- Generowanie czytelnych i estetycznych partytur PDF, które będą użyteczne dla muzyków.

Rozwiązanie tego problemu ma potencjał znaczącego usprawnienia pracy muzyków, ułatwienia analizy muzycznej oraz wsparcia procesów edukacyjnych w dziedzinie muzyki fortepianowej.

2.2 Osadzenie tematu w kontekście aktualnego stanu wiedzy

Automatyczna transkrypcja muzyki (ATM, ang. *Automatic Music Transcription*) jest jednym z najbardziej złożonych problemów w dziedzinie przetwarzania sygnałów muzycznych (MIR, ang. *Music Information Retrieval*). W kontekście muzyki fortepianowej, ATM zyskuje szczególne znaczenie ze względu na popularność tego instrumentu oraz jego rolę w edukacji muzycznej i kompozycji.

Obecny stan wiedzy w dziedzinie ATM dla fortepianu charakteryzuje się następującymi aspektami:

1. **Wykorzystanie głębokiego uczenia:** Współczesne podejścia do ATM fortepianowego opierają się głównie na zastosowaniu głębokich sieci neuronowych. Modele takie jak konwolucyjne sieci neuronowe (CNN) i rekurencyjne sieci neuronowe (RNN) wykazały znaczącą skuteczność w rozpoznawaniu struktur muzycznych w spektrogramach audio [3].
2. **Modele hybrydowe:** Najnowsze badania skupiają się na modelach hybrydowych, łączących różne architektury sieci neuronowych. Na przykład, model "Onsets and Frames"[3] wykorzystuje osobne sieci do detekcji początków dźwięków i ich trwania, co znacząco poprawiło dokładność transkrypcji.
3. **Przetwarzanie w czasie rzeczywistym:** Rozwój technik przetwarzania sygnałów i optymalizacji algorytmów pozwala na coraz szybszą transkrypcję, zbliżającą się do przetwarzania w czasie rzeczywistym [6].
4. **Modelowanie akustyki fortepianu:** Zaawansowane modele uwzględniają specyfikę akustyki fortepianu, w tym zjawiska takie jak rezonans strun i wpływ pedałów na brzmienie [2].

5. **Transkrypcja ekspresji muzycznej:** Nowsze badania skupiają się nie tylko na rozpoznawaniu nut, ale także na odwzorowaniu dynamiki, artykulacji i innych aspektów ekspresji muzycznej [5].

Mimo znaczących postępów w dziedzinie ATM dla fortepianu, wciąż istnieją pewne ograniczenia i wyzwania:

- **Dokładność w muzyce polifonicznej:** Transkrypcja złożonych struktur polifonicznych, charakterystycznych dla zaawansowanej literatury fortepianowej, wciąż stanowi wyzwanie dla obecnych systemów [1].
- **Rozpoznawanie bardzo cichych dźwięków:** Detekcja nut granych bardzo cicho, szczególnie w kontekście głośniejszego akompaniamentu, pozostaje problematyczna [8].
- **Odróżnianie bliskich harmonicznie dźwięków:** W gęstych strukturach harmonicznych, rozróżnienie blisko brzmiących dźwięków może być trudne dla systemów ATM [1].
- **Interpretacja rytmiczna:** Precyzyjne odwzorowanie złożonych struktur rytmicznych, szczególnie w muzyce z rubato lub improwizowanej, stanowi wyzwanie dla obecnych systemów [2].
- **Wykorzystanie pedałów:** Uwzględnienie u użytkownika pedałów fortepianowych, które wpływają na rezonans strun i brzmienie, wymaga zaawansowanych technik analizy akustycznej [2].

Projekt TuneScript wpisuje się w ten kontekst, stawiając sobie za cel adresowanie wymienionych wyzwań poprzez integrację zaawansowanego modelu AI do transkrypcji z MP3 do MIDI z narzędziami do generowania partytur PDF. Takie kompleksowe podejście ma potencjał przezwyciężenia niektórych ograniczeń obecnych rozwiązań i dostarczenia użytkownikom narzędzia o wysokiej praktycznej wartości.

2.3 Studia literaturowe

W celu głębszego zrozumienia problematyki automatycznej transkrypcji muzyki fortepianowej oraz określenia aktualnego stanu wiedzy w tej dziedzinie, przeprowadzono szczegółowy przegląd literatury. Poniżej przedstawiono kluczowe prace i ich wkład w rozwój tej dziedziny:

1. **Kong et al. (2020)** [7] w pracy "High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times" prezentują nowatorskie podejście do transkrypcji fortepianowej, które stanowi podstawę modelu wykorzystywanego w projekcie TuneScript. Autorzy proponują metodę wysokiej rozdzielczości czasowej do transkrypcji fortepianowej, która uwzględnia nie tylko detekcję nut, ale także użycie pedałów. Kluczową innowacją jest zastosowanie regresji do precyzyjnego określania czasów początku (onset) i końca (offset) każdej nuty, co pozwala na dokładniejsze odwzorowanie rytmu i artykulacji. Model ten wykazuje znaczącą poprawę w stosunku do wcześniejszych rozwiązań, szczególnie w zakresie precyzji czasowej i detekcji użycia pedałów.
2. **Benetos et al. (2013)** [1] w swojej pracy przeglądowej "Automatic music transcription: challenges and future directions" przedstawiają kompleksowe omówienie technik ATM. Autorzy omawiają główne wyzwania w dziedzinie, takie jak polifonia, różnorodność instrumentów i style wykonania. W kontekście muzyki fortepianowej, praca ta podkreśla złożoność problemu transkrypcji ze względu na szeroką gamę dynamiczną i bogactwo harmoniczne fortepianu.
3. **Hawthorne et al. (2018)** [3] w artykule "Onsets and frames: Dual-objective piano transcription" prezentują przełomowy model do transkrypcji fortepianowej. Ich podejście, nazwane "Onsets and Frames", wykorzystuje osobne sieci neuronowe do detekcji początków dźwięków i ich trwania. Model ten znacząco poprawił dokładność transkrypcji fortepianowej, osiągając wyniki bliskie ludzkiej dokładności w niektórych aspektach. Jest to jedno z najbardziej zaawansowanych rozwiązań w dziedzinie ATM dla fortepianu, które stanowiło inspirację dla późniejszych prac, w tym modelu Kong et al.
4. **Kelz et al. (2016)** [6] w pracy "On the Potential of Simple Framewise Approaches to Piano Transcription" badają możliwości prostszych podejść do transkrypcji fortepianowej, opierających się na analizie pojedynczych ramek czasowych. Autorzy wykazują, że nawet stosunkowo proste modele mogą osiągać konkurencyjne wyniki, co ma istotne implikacje dla projektowania systemów działających w czasie rzeczywistym.
5. **Cheng et al. (2016)** [2] w artykule "Attack/decay analysis of piano sounds using an exponential model" skupiają się na modelowaniu akustycznych właściwości dźwięków fortepianowych. Ich praca jest istotna dla zrozumienia, jak precyzyjnie analizować początkową fazę dźwięku (atak) oraz jego wygaszanie (decay), co jest kluczowe dla dokładnej transkrypcji dynamiki i artykulacji.
6. **Jeong et al. (2017)** [5] w pracy "Deep Learning for Music" prezentują szerokie zastosowanie technik głębokiego uczenia w różnych zadaniach związanych z prze-

tworzeniem muzyki, w tym w transkrypcji fortepianowej. Autorzy omawiają, jak różne architektury sieci neuronowych mogą być wykorzystane do modelowania różnych aspektów muzyki, od wysokości dźwięku po ekspresję.

7. **Sigtia et al. (2016)** [8] w pracy "An end-to-end neural network for polyphonic piano music transcription" prezentują kompleksowe podejście do transkrypcji muzyki fortepianowej wykorzystujące głębokie sieci neuronowe. Ich model jest w stanie bezpośrednio przetwarzać surowy sygnał audio na notację muzyczną, co stanowi znaczący krok w kierunku w pełni automatycznej transkrypcji.

Powyższe prace stanowią fundament teoretyczny dla projektu TuneScript. Pokazują one ewolucję technik ATM dla fortepianu od tradycyjnych metod przetwarzania sygnałów do zaawansowanych modeli uczenia głębokiego. Jednocześnie, literatura ta wskazuje na wciąż istniejące wyzwania, szczególnie w zakresie transkrypcji muzyki polifonicznej, dokładnego odwzorowania dynamiki i ekspresji oraz generowania wysokiej jakości partytur.

Projekt TuneScript czerpie z tych badań, integrując najnowsze osiągnięcia w dziedzinie ATM z praktycznym aspektem generowania użytecznych partytur. Szczególnie istotna jest praca Kong et al. [7], która stanowi podstawę modelu transkrypcji wykorzystywanego w systemie. Model ten, dzięki swojej wysokiej rozdzielczości czasowej i uwzględnieniu pedałów, pozwala na bardziej precyzyjną transkrypcję niż wcześniejsze rozwiązania.

Warto zauważyć, że mimo znaczącego postępu w dziedzinie ATM dla fortepianu, wciąż istnieje przestrzeń do dalszych ulepszeń, szczególnie w zakresie dokładności transkrypcji bardzo złożonych utworów polifonicznych oraz wiernego odwzorowania subtelności ekspresji muzycznej. TuneScript stara się adresować te wyzwania, oferując kompleksowe rozwiązanie łączące zaawansowaną transkrypcję z praktycznym zastosowaniem w postaci generowania partytur.

Rozdział 3

Wymagania i narzędzia

Rozdział ten przedstawia przegląd wymagań i narzędzi niezbędnych do realizacji systemu TuneScript. Omówione zostaną zarówno wymagania funkcjonalne i нефункционалне, jak również wymagania sprzętowe i programowe oraz wykorzystane technologie i narzędzia. Szczególną uwagę poświęcono doborowi odpowiednich komponentów i bibliotek, które umożliwiają efektywną implementację złożonych funkcjonalności systemu.

3.1 Komponenty, moduły i biblioteki

Realizacja systemu TuneScript wymaga wykorzystania szeregu nowoczesnych technologii i narzędzi programistycznych. Wybór konkretnych rozwiązań został podyktowany specyficznymi wymaganiami projektu, takimi jak wydajne przetwarzanie dźwięku, skalowalność systemu oraz zapewnienie intuicyjnego interfejsu użytkownika. Poniżej przedstawiono główne komponenty i biblioteki wraz z uzasadnieniem ich wyboru:

- **Frontend:**
 - Next.js: Framework React wybrany ze względu na:
 - * Wydajne renderowanie po stronie serwera (SSR).
 - * Wbudowaną optymalizację SEO.
 - * Automatyczny routing i code splitting.
 - * Wsparcie dla API Routes ułatwiające integrację z backendem.
 - TypeScript: Rozszerzenie JavaScript zastosowane dla:
 - * Wczesnego wykrywania błędów dzięki statycznemu typowaniu.
 - * Lepszej dokumentacji kodu poprzez interfejsy i typy.
 - * Udoskonalonego wsparcia IDE i autouzupełniania.
 - Apollo Client: Klient GraphQL wybrany ze względu na:
 - * Zaawansowane zarządzanie stanem aplikacji.

- * Inteligentne cachowanie zapytań.
- * Wsparcie dla TypeScript.

- **Backend:**

- Python: Główny język programowania wybrany ze względu na:
 - * Rozbudowany ekosystem bibliotek do uczenia maszynowego.
 - * Wydajne przetwarzanie danych numerycznych.
 - * Czytelną składnię ułatwiającą rozwój projektu.
- Django: Framework webowy zastosowany dla:
 - * Solidnego systemu autentykacji i autoryzacji.
 - * ORM ułatwiającego pracę z bazą danych.
 - * Wbudowanych mechanizmów bezpieczeństwa.
- Celery z Redis: System kolejkowania wybrany dla:
 - * Asynchronicznego przetwarzania długotrwałych zadań.
 - * Efektywnej dystrybucji obciążenia.
 - * Niezawodnej komunikacji między komponentami.

- **Komponenty ML:**

- PyTorch: Framework ML wybrany ze względu na:
 - * Elastyczność w definiowaniu modeli neuronowych.
 - * Wydajne przetwarzanie na GPU.
 - * Rozbudowane narzędzia do treningu modeli.
- Transformers: Biblioteka NLP zastosowana dla:
 - * Dostępu do najnowszych modeli językowych.
 - * Gotowych implementacji architektury Transformer.
 - * Łatwej integracji z PyTorch.

- **Infrastruktura:**

- PostgreSQL: System bazodanowy wybrany ze względu na:
 - * Wsparcie dla złożonych zapytań i indeksów.
 - * Niezawodność i skalowalność.
 - * Dobrą integrację z Django.
- Docker z Compose: Narzędzia konteneryzacji zastosowane dla:
 - * Spójnego środowiska deweloperskiego.

- * Łatwego zarządzania mikrousługami.
- * Uproszczonego wdrażania aplikacji.

Powyższa struktura technologiczna została zaprojektowana z myślą o zapewnieniu wydajności, skalowalności i łatwości utrzymania systemu. Wybrane narzędzia reprezentują sprawdzone rozwiązania w swoich dziedzinach, jednocześnie oferując nowoczesne podejście do tworzenia aplikacji webowych wspierających uczenie maszynowe. Dla zapewnienia jakości kodu i ułatwienia pracy zespołowej, projekt wykorzystuje szereg narzędzi deweloperskich:

- **Zarządzanie zależnościami i jakość kodu:**
 - Poetry: Nowoczesne zarządzanie zależnościami Python.
 - ESLint i Prettier: Spójne formatowanie kodu JavaScript/TypeScript.
 - Black i isort: Automatyczne formatowanie kodu Python.
 - Flake8: Analiza statyczna kodu Python.
- **Testowanie:**
 - pytest: Kompleksowe testowanie komponentów Pythona.
 - Jest: Testy jednostkowe i integracyjne frontendu.
 - Factory Boy: Generowanie danych testowych.

3.2 Wymagania funkcjonalne

Wymagania funkcjonalne systemu TuneScript definiują konkretne funkcje i możliwości, które system musi zapewniać użytkownikom. Zostały one opracowane na podstawie analizy potrzeb docelowych użytkowników oraz specyfiki procesu transkrypcji muzycznej. System musi zapewniać kompletny zestaw funkcjonalności umożliwiających zarówno pracę z transkrypcjami, jak i interakcję między użytkownikami. Poniżej przedstawiono szczegółowy wykaz wymagań funkcjonalnych:

1. Uwierzytelnianie i autoryzacja użytkowników:

- System musi zapewniać funkcjonalności rejestracji, logowania i wylogowywania użytkowników.
- Użytkownicy muszą mieć możliwość resetowania haseł za pomocą poczty elektronicznej.

2. Przesyłanie plików:

- System musi umożliwiać użytkownikom przesyłanie plików audio w formatach MP3 i WAV.
- System musi weryfikować format przesyłanych plików.

3. Przetwarzanie transkrypcji:

- System musi przetwarzać przesłane pliki audio w celu generowania transkrypcji fortepianowych.
- Transkrypcje powinny być zapisywane w formacie MIDI oraz generowane jako partytury PDF.

4. Zarządzanie transkrypcjami:

- Użytkownicy muszą mieć możliwość przeglądania, edytowania i usuwania swoich transkrypcji.
- Użytkownicy muszą mieć możliwość ustawiania widoczności swoich transkrypcji jako publicznych lub prywatnych.

5. Funkcjonalność wyszukiwania:

- System musi zapewniać możliwości wyszukiwania transkrypcji na podstawie tytułu, kompozytora i widoczności.

6. System oceniania:

- Użytkownicy muszą mieć możliwość oceniania transkrypcji.
- System musi obliczać i wyświetlać średnią ocenę dla każdej transkrypcji.

7. Zarządzanie ulubionymi:

- Użytkownicy muszą mieć możliwość dodawania transkrypcji do ulubionych.
- Użytkownicy muszą mieć możliwość przeglądania i zarządzania swoimi ulubionymi transkrypcjami.

3.3 Wymagania нефunkcjonalne

W celu zapewnienia wysokiej jakości działania systemu, oprócz wymagań funkcjonalnych, zdefiniowano szereg wymagań нефunkcjonalnych. Określają one cechy jakościowe systemu, takie jak wydajność, niezawodność czy bezpieczeństwo. Poniżej przedstawiono kluczowe wymagania нефunkcjonalne:

1. **Wydajność:** Wydajność systemu jest kluczowa dla zapewnienia dobrego doświadczenia użytkownika. System powinien spełniać następujące kryteria:

- System musi przetwarzać transkrypcje w rozsądnych ramach czasowych (np. mniej niż 5 minut dla 10-minutowego pliku audio).
- System powinien obsługiwać jednocześnie co najmniej 100 aktywnych użytkowników.

2. Użyteczność i doświadczenie użytkownika:

- Interfejs użytkownika musi być intuicyjny i łatwy w nawigacji.
- System musi dostarczać użytkownikom jasne instrukcje i informacje zwrotne.

3. Niezawodność i dostępność:

- System musi być niezawodny, z dostępnością na poziomie 99,9% (nie licząc planowanych przerw technicznych).
- System musi zapewniać mechanizmy automatycznego odzyskiwania po awariach.

4. Bezpieczeństwo i prywatność:

- System musi chronić dane użytkowników za pomocą szyfrowania.
- System musi wykorzystywać bezpieczne mechanizmy uwierzytelniania.

5. Kompatybilność i dostępność:

- System musi być kompatybilny z najnowszymi wersjami przeglądarek Chrome, Firefox, Safari i Edge.
- Interfejs użytkownika powinien być responsywny i działać na urządzeniach mobilnych.

3.4 Wymagania sprzętowe i programowe

Prawidłowe funkcjonowanie systemu TuneScript wymaga spełnienia określonych wymagań technicznych, zarówno po stronie klienta, jak i serwera. Wymagania te zostały zoptymalizowane tak, aby zapewnić szeroką dostępność systemu przy jednoczesnym zachowaniu wysokiej jakości działania.

3.4.1 Wymagania dla użytkowników końcowych

System został zaprojektowany z myślą o minimalnych wymaganiach sprzętowych dla użytkowników końcowych:

- Urządzenie z dostępem do internetu (komputer, tablet lub smartfon).

- Nowoczesna przeglądarka internetowa (Chrome, Firefox, Safari lub Edge w najnowszej wersji).
- Minimum 4 GB RAM dla optymalnego działania aplikacji.
- Stabilne połączenie internetowe o prędkości co najmniej 5 Mbps.

3.4.2 Wymagania dla środowiska serwerowego

Dla zapewnienia płynnej pracy systemu, serwer musi spełniać następujące wymagania:

- Procesor: minimum 4 rdzenie, zalecane 8 rdzeni.
- RAM: minimum 16 GB, zalecane 32 GB.
- Dysk: minimum 100 GB przestrzeni dyskowej SSD.
- System operacyjny: Linux (zalecany Ubuntu 20.04 LTS lub nowszy).
- Stabilne połączenie internetowe o wysokiej przepustowości.

3.5 Przypadki użycia

Zdefiniowane przypadki użycia przedstawiają typowe interakcje użytkowników z systemem TuneScript. Dokumentacja przypadków użycia pozwala na lepsze zrozumienie sposobu, w jaki różni użytkownicy będą korzystać z systemu oraz jakie funkcjonalności muszą być dostępne dla poszczególnych grup użytkowników.

Kluczowe przypadki użycia obejmują:

- **Rejestracja i uwierzytelnianie:** Obejmuje proces tworzenia konta, logowania i zarządzania dostępem do systemu.
- **Zarządzanie transkrypcjami:** Dotyczy przesyłania plików audio, generowania transkrypcji oraz zarządzania wygenerowanymi partyturami.
- **Interakcje społecznościowe:** Obejmuje ocenianie transkrypcji, dodawanie ich do ulubionych oraz udostępnianie innym użytkownikom.
- **Wyszukiwanie i przeglądanie:** Dotyczy możliwości przeszukiwania bazy transkrypcji i przeglądania dostępnych materiałów.
- **Zarządzanie kontem:** Obejmuje funkcje związane z personalizacją ustawień konta i preferencji użytkownika.

Rozdział 4

Specyfikacja zewnętrzna

Rozdział ten przedstawia specyfikację zewnętrzną systemu TuneScript, opisując interfejsy użytkownika, scenariusze korzystania z systemu oraz wymagania dotyczące integracji z innymi systemami. Specyfikacja zewnętrzna ma na celu zaprezentowanie funkcjonalności systemu z perspektywy użytkownika, uwzględniając różne przypadki użycia i interakcje z systemem.

4.1 Wymagania sprzętowe i programowe

System TuneScript został zaprojektowany jako aplikacja webowa działająca w architekturze klient-serwer. Poniżej przedstawiono minimalne i zalecane wymagania sprzętowe oraz programowe niezbędne do prawidłowego funkcjonowania systemu.

4.1.1 Wymagania programowe

Do uruchomienia systemu TuneScript wymagane jest odpowiednie środowisko programowe. Na serwerze konieczny jest system operacyjny Linux (zalecany Ubuntu 20.04 LTS lub nowszy) wraz z zainstalowanym Dockerem w wersji 20.10 lub nowszej oraz Docker Compose w wersji 1.29 lub nowszej. Po stronie klienta wymagana jest jedynie nowoczesna przeglądarka internetowa, taka jak Chrome 90+, Firefox 88+, Safari 14+ lub Edge 90+.

4.2 Kategorie użytkowników

System TuneScript przewiduje trzy główne kategorie użytkowników, różniące się poziomem dostępu i możliwościami w systemie. Niezarejestrowani użytkownicy mogą przeglądać jedynie publiczne transkrypcje. Zarejestrowani użytkownicy otrzymują dostęp do pełnej funkcjonalności systemu, włączając możliwość przesyłania plików audio, generowania transkrypcji oraz zarządzania własnymi materiałami. Administratorzy posiadają

pełny dostęp do systemu, w tym do panelu administracyjnego, umożliwiającego zarządzanie użytkownikami i konfiguracją systemu.

4.3 Sposób obsługi

Korzystanie z systemu TuneScript zostało zaprojektowane w sposób intuicyjny i przyjazny dla użytkownika. Poniżej przedstawiono główne funkcjonalności systemu oraz sposób ich obsługi:

1. Rejestracja i logowanie:

- Użytkownik może zarejestrować się, podając nazwę użytkownika, adres e-mail i hasło.
- Następnie otrzymuje e-mail z linkiem potwierdzającym.
- Po potwierdzeniu, użytkownik może zalogować się do systemu.

2. Przesyłanie pliku audio:

- Zalogowany użytkownik może przesłać plik audio w formacie MP3 lub WAV.
- System weryfikuje format pliku.

3. Generowanie transkrypcji:

- Po przesłaniu pliku, system automatycznie rozpoczyna proces transkrypcji.
- Użytkownik jest informowany o postępie procesu.

4. Zarządzanie transkrypcjami:

- Użytkownik może przeglądać, edytować i usuwać swoje transkrypcje.
- Możliwe jest ustawienie widoczności transkrypcji jako publicznej lub prywatnej.

5. Wyszukiwanie i przeglądanie transkrypcji:

- Użytkownicy mogą wyszukiwać transkrypcje po tytule, kompozytorze i widoczności.
- Możliwe jest przeglądanie publicznych transkrypcji innych użytkowników.

6. Ocenianie i dodawanie do ulubionych:

- Zalogowani użytkownicy mogą oceniać transkrypcje.
- Możliwe jest dodawanie transkrypcji do listy ulubionych.

4.4 Administracja systemem

Panel administracyjny systemu TuneScript, zbudowany w oparciu o framework Django, zapewnia administratorom pełną kontrolę nad funkcjonowaniem aplikacji. Administrator ma możliwość zarządzania kontami użytkowników, moderowania treści oraz monitorowania aktywności systemu. Panel umożliwia również konfigurację kluczowych ustawień systemu i generowanie raportów dotyczących jego wykorzystania. Do głównych zadań administracyjnych należy weryfikacja nowych użytkowników, monitorowanie jakości transkrypcji oraz zarządzanie zasobami systemowymi.

4.5 Kwestie bezpieczeństwa

Bezpieczeństwo danych i prywatność użytkowników stanowią kluczowy aspekt systemu TuneScript. Wszystkie dane przesyłane między klientem a serwerem są szyfrowane przy użyciu protokołu HTTPS, a hasła użytkowników przechowywane są w formie zaszyfrowanej z wykorzystaniem nowoczesnych algorytmów haszujących. System implementuje mechanizm uwierzytelniania oparty na tokenach JWT oraz zaawansowane zabezpieczenia przed popularnymi rodzajami ataków, takimi jak CSRF i XSS. Regularnie przeprowadzane są aktualizacje wszystkich wykorzystywanych bibliotek i zależności, co pozwala na utrzymanie wysokiego poziomu bezpieczeństwa. System został również wyposażony w mechanizmy monitorowania i wykrywania potencjalnych zagrożeń.

4.6 Przykład działania

Dla lepszego zrozumienia funkcjonowania systemu, poniżej przedstawiono typowy scenariusz użycia aplikacji TuneScript:

1. Użytkownik rozpoczyna od rejestracji w systemie, podając wymagane dane osobowe.
2. Po potwierdzeniu rejestracji poprzez link aktywacyjny, użytkownik loguje się do systemu.
3. Następnie przesyła plik audio zawierający nagranie fortepianowe w jednym z obsługiwanych formatów.
4. System automatycznie rozpoczyna proces przetwarzania pliku i generowania transkrypcji.
5. W czasie rzeczywistym wyświetlany jest postęp procesu transkrypcji.
6. Po zakończeniu procesu, użytkownik otrzymuje dostęp do wygenerowanej transkrypcji.

7. Transkrypcję można przeglądać, edytować parametry oraz pobierać w różnych formatach.
8. Użytkownik może zdecydować o udostępnieniu transkrypcji publicznie.
9. Inni użytkownicy mogą wyszukiwać, przeglądać i oceniać udostępnioną transkrypcję.

Rozdział 5

Specyfikacja wewnętrzna

Rozdział ten przedstawia szczegółową specyfikację wewnętrzną systemu TuneScript. Omówione zostaną aspekty architektury systemu, wykorzystane technologie oraz kluczowe decyzje implementacyjne. Szczególna uwaga zostanie poświęcona architekturze mikroserwisowej, która została wybrana jako fundament systemu, strukturom danych oraz algorytmom wykorzystanym w procesie transkrypcji muzycznej.

5.1 Przedstawienie idei

TuneScript to zaawansowana aplikacja webowa, która automatyzuje proces transkrypcji nagrań fortepianowych w formacie MP3 na partytury w formacie PDF. Główna idea projektu opiera się na wykorzystaniu sztucznej inteligencji, a dokładniej głębokich sieci neuronowych, do analizy i interpretacji sygnału audio, a następnie przekształcenia go w notację muzyczną.

Kluczowe aspekty idei TuneScript obejmują:

- Wykorzystanie zaawansowanych technik przetwarzania sygnałów audio do precyzyjnej detekcji nut.
- Implementację modelu głębokiego uczenia do transkrypcji muzyki fortepianowej.
- Integrację procesu transkrypcji z systemem generowania partytur.
- Stworzenie przyjaznego dla użytkownika interfejsu webowego do zarządzania transkrypcjami.

5.2 Architektura systemu

TuneScript wykorzystuje architekturę mikroserwisową (Microservices Architecture), która umożliwia niezależne rozwijanie, wdrażanie i skalowanie poszczególnych komponentów systemu. Architektura składa się z następujących serwisów:

1. **Serwis frontendowy** jest implementowany w Next.js i TypeScript. Głównym zadaniem tego serwisu jest zapewnienie responsywnego interfejsu użytkownika. Komunikacja z backendem odbywa się poprzez GraphQL API. Serwis ten jest również odpowiedzialny za monitorowanie postępu transkrypcji poprzez SSE.
2. **Serwis backendowy** został zbudowany przy użyciu Django z GraphQL (Graphene). Jest odpowiedzialny za zarządzanie logiką biznesową aplikacji oraz autentykację i autoryzację użytkowników. Serwis koordynuje również proces transkrypcji i generowania partytur.
3. **Serwis pracownika (worker)** bazuje na technologii Celery i jest odpowiedzialny za przetwarzanie zadań asynchronicznych. Jego głównym zadaniem jest wykonywanie transkrypcji plików audio do formatu MIDI oraz generowanie partytur PDF z plików MIDI.
4. **Serwis chatbota** jest implementowany przy użyciu Flask. Wykorzystuje architekturę RAG do obsługi zapytań użytkowników i zapewnia wsparcie w czasie rzeczywistym. Działa jako niezależny serwis pomocniczy, wspierający interakcję z użytkownikiem.

Komponenty infrastrukturalne:

1. Serwisy przechowywania:

- PostgreSQL jako główna baza danych
- Redis do cachowania i kolejkowania zadań
- System plików do przechowywania plików multimedialnych

Zastosowana architektura mikroservisowa zapewnia następujące korzyści:

- **Modularność:** Każdy serwis jest niezależnym komponentem
- **Skalowalność:** Możliwość skalowania poszczególnych serwisów według potrzeb
- **Niezawodność:** Awaria jednego serwisu nie wpływa na działanie pozostałych
- **Elastyczność technologiczna:** Możliwość wyboru optymalnych technologii dla każdego serwisu
- **Efektywność rozwoju:** Łatwiejsze zarządzanie kodem i wdrożeniami

Całość systemu jest konteneryzowana przy użyciu Docker, a orkiestracja serwisów odbywa się za pomocą Docker Compose, co zapewnia spójne środowisko zarówno dla rozwoju, jak i wdrożenia.

5.3 Opis struktur danych i organizacji bazy danych

TuneScript wykorzystuje relacyjną bazę danych PostgreSQL do przechowywania danych. Schemat bazy danych przedstawiony jest na Rysunku 5.2. Główne modele w bazie danych to:

- **User:** Rozszerzenie standardowego modelu użytkownika Django, zawierające dodatkowe pole do potwierdzenia e-maila.
- **Profile:** Przechowuje dodatkowe informacje o użytkowniku, takie jak biografia, zdjęcie profilowe, preferencje oraz informacje o statusie premium.
- **AudioFile:** Zawiera informacje o przesłanych plikach audio, w tym tytuł i ścieżkę do pliku.
- **Transcription:** Przechowuje metadane transkrypcji, w tym odniesienia do pliku audio, użytkownika, tagów, oraz informacje o statusie, ocenach i widoczności.
- **Rating:** Zawiera oceny użytkowników dla poszczególnych transkrypcji.
- **Favorite:** Przechowuje informacje o ulubionych transkrypcjach użytkowników.
- **MIDIFile:** Zawiera informacje o wygenerowanych plikach MIDI dla transkrypcji.
- **SheetMusic:** Przechowuje informacje o wygenerowanych partyturach PDF dla transkrypcji.
- **Tag:** Zawiera tagi, które mogą być przypisane do transkrypcji.
- **UserPlayHistory:** Przechowuje historię odtwarzania transkrypcji przez użytkowników, w tym czas odtwarzania i liczbę odtworzeń.

Modele te są ze sobą powiązane za pomocą kluczy obcych, tworząc kompleksową strukturę danych, która umożliwia efektywne zarządzanie informacjami w systemie TuneScript. Jak widać na Rysunku 5.2, szczególną uwagę zwrócono na optymalizację wydajności poprzez zastosowanie indeksów oraz unikalnych ograniczeń tam, gdzie jest to konieczne.

5.4 Skrypty deweloperskie

Projekt TuneScript zawiera szereg skryptów ułatwiających proces rozwoju i testowania. Są one zdefiniowane w pliku `package.json`:

Listing 5.1: Skrypty deweloperskie

```
1 {
```

```
2  "name": "tunescript-devtools",
3  "version": "1.0.0",
4  "description": "Development tools for TuneScript",
5  "scripts": {
6    "start": "docker-compose up -d",
7    "stop": "docker-compose down -v",
8    "start:persistent": "docker-compose -f docker-compose.
      persistent.yml up -d",
9    "stop:persistent": "docker-compose -f docker-compose.
      persistent.yml down",
10   "start:fresh": "docker-compose -f docker-compose.fresh.yml up
      -d",
11   "stop:fresh": "docker-compose -f docker-compose.fresh.yml
      down -v",
12   "restart": "npm run stop && npm run start",
13   "restart:persistent": "npm run stop:persistent && npm run
      start:persistent",
14   "restart:fresh": "npm run stop:fresh && npm run start:fresh",
15   "logs": "docker-compose logs -f",
16   "test": "docker-compose exec frontend npm run test",
17   "test:watch": "docker-compose exec frontend npm run test:
      watch",
18   "test:coverage": "docker-compose exec frontend npm run test:
      coverage",
19   "test:backend": "docker-compose exec web poetry run pytest
      tunescript_app/tests",
20   "format": "docker-compose exec web /app/format.sh"
21 }
22 }
```

Te skrypty umożliwiają łatwe uruchamianie, zatrzymywanie i restartowanie aplikacji w różnych trybach, a także uruchamianie testów i formatowanie kodu.

5.5 Przegląd ważniejszych algorytmów

Kluczowym algorytmem w systemie TuneScript jest model transkrypcji muzycznej oparty na pracy Kong et al. (2020) "High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times". Główne etapy tego algorytmu to:

1. Przetwarzanie wstępne sygnału audio (np. normalizacja, konwersja do spektrogramu).

2. Analiza sygnału przez sieć konwolucyjną w celu wykrycia początków (onset) i końców (offset) nut.
3. Regresja czasów początku i końca każdej nuty.
4. Detekcja użycia pedałów fortepianu.
5. Generowanie transkrypcji MIDI na podstawie wykrytych nut i pedałów.

5.6 Szczegóły implementacji wybranych fragmentów

W tej sekcji przedstawiono szczegółową implementację kluczowych elementów systemu TuneScript. Skupiono się na trzech głównych aspektach: modelu transkrypcji muzycznej, systemie kolejkovania zadań oraz mechanizmie monitorowania postępu transkrypcji w czasie rzeczywistym. Każdy z tych elementów stanowi istotną część systemu i wymaga specyficznego podejścia implementacyjnego.

5.6.1 Implementacja modelu transkrypcji

Model transkrypcji fortepianowej wykorzystuje zaawansowaną architekturę konwolucyjno-rekurencyjną do detekcji nut. Architektura modelu opiera się na kilku kluczowych komponentach przetwarzających sygnał audio. Pierwszym z nich jest ekstraktor spektrogramu, który przekształca surowy sygnał audio na reprezentację czasowo-częstotliwościową. Następnie, ekstraktor cech mel-spektrogramu konwertuje spektrogram na skalę mel, która lepiej odpowiada ludzkiemu postrzeganiu dźwięku. Główny element modelu stanowią cztery równoległe sieci CNN-RNN, z których każda specjalizuje się w detekcji innego aspektu dźwięku: ramek czasowych, początków nut, końców nut oraz głośności. Całość uzupełnia dwukierunkowa sieć rekurencyjna GRU, która przetwarza sekwencyjnie otrzymane cechy, uwzględniając kontekst czasowy wykonania utworu.

Listing 5.2: Model regresji dla momentu naciśnięcia i puszczenia klawisza

```

1  class RegressOnsetOffsetFrameVelocityCRNN(nn.Module):
2      def __init__(self, frames_per_second, classes_num):
3          super().__init__()
4
5          window_size = 2048
6          hop_size = sample_rate // frames_per_second
7          mel_bins = 229
8
9          # Ekstraktor cech spektralnych
10         self.spectrogram_extractor = Spectrogram(
11             n_fft=window_size,
12             hop_length=hop_size,
```

```

13         win_length=window_size ,
14         window='hann' ,
15         center=True ,
16         pad_mode='reflect' ,
17         freeze_parameters=True)
18
19     # Ekstraktor cech mel-spektrogramu
20     self.logmel_extractor = LogmelFilterBank(
21         sr=sample_rate ,
22         n_fft=window_size ,
23         n_mels=mel_bins ,
24         fmin=30 ,
25         fmax=sample_rate // 2 ,
26         freeze_parameters=True)
27
28     # Modele do detekcji różnych aspektów
29     self.frame_model = AcousticModelCRnn8Dropout(
30         classes_num , midfeat , momentum)
31     self.reg_onset_model = AcousticModelCRnn8Dropout(
32         classes_num , midfeat , momentum)
33     self.reg_offset_model = AcousticModelCRnn8Dropout(
34         classes_num , midfeat , momentum)
35     self.velocity_model = AcousticModelCRnn8Dropout(
36         classes_num , midfeat , momentum)
37
38     # GRU do przetwarzania sekwencyjnego
39     self.reg_onset_gru = nn.GRU(
40         input_size=88 * 2 ,
41         hidden_size=256 ,
42         num_layers=1 ,
43         batch_first=True ,
44         bidirectional=True)

```

5.6.2 Implementacja systemu kolejkowania transkrypcji

System kolejkowania zadań został zaprojektowany do obsługi długotrwałego procesu transkrypcji w sposób asynchroniczny. Proces rozpoczyna się od pobrania i przygotowania pliku audio do analizy. Następnie wykonywana jest właściwa transkrypcja przy użyciu modelu uczenia maszynowego, którego wyniki są przekształcane w plik MIDI. W kolejnym kroku system generuje partytury w formacie PDF na podstawie utworzonego pliku MIDI. Cały proces jest monitorowany, a jego status jest na bieżąco aktualizowany w bazie danych. Implementacja wykorzystuje framework Celery do zarządzania zadaniami, zapewniając mechanizm ponownych prób w przypadku wystąpienia błędów, co zwiększa niezawodność systemu.

Listing 5.3: Asynchroniczne zadanie do transkrypcji

```

1 @shared_task(bind=True, max_retries=3)
2 def process_transcription(self, transcription_id):
3     try:
4         transcription = Transcription.objects.get(pk=transcription_id)
5         audio_file = transcription.audio_file
6
7         # Wczytanie i preprocessing audio
8         audio, _ = load_audio(
9             audio_file.audio_file.path,
10             sr=sample_rate,
11             mono=True)
12
13         # Inicjalizacja modelu transkrypcji
14         transcriptor = PianoTranscription(device='cpu')
15
16         # Generowanie pliku MIDI
17         with tempfile.NamedTemporaryFile(
18             delete=False, suffix='.mid') as tmp_midi:
19             midi_path = tmp_midi.name
20             transcriptor.transcribe(audio, midi_path)
21
22         # Zapisywanie pliku MIDI
23         with open(midi_path, 'rb') as midi_file:
24             midi_instance = MIDIFile(transcription=transcription)
25             midi_instance.midi_file.save(
26                 f"{transcription.title}.mid",
27                 File(midi_file))
28
29         # Generowanie PDF z MIDI
30         pdf_path = os.path.splitext(midi_path)[0] + '.pdf'
31         convert_midi_to_pdf(midi_data, pdf_path)
32
33         # Zapisywanie pliku PDF
34         with open(pdf_path, 'rb') as pdf_file:
35             sheet_music = SheetMusic(transcription=transcription)
36             sheet_music.pdf_file.save(
37                 f"{transcription.title}.pdf",
38                 File(pdf_file))
39
40         transcription.status = 'COMPLETED'
41         transcription.save()
42
43     except Exception as e:
44         transcription.status = 'FAILED'
45         transcription.save()
46         raise self.retry(exc=e, countdown=60)

```

5.6.3 Implementacja systemu SSE do monitorowania postępu

Mechanizm Server-Sent Events (SSE) umożliwia przekazywanie informacji o postępie transkrypcji do interfejsu użytkownika w czasie rzeczywistym. Po zainicjowaniu połączenia, system ustanawia długotrwałe połączenie HTTP z przeglądarką klienta. W regularnych odstępach czasu sprawdzany jest status transkrypcji w bazie danych, a wszelkie zmiany są natychmiast przesyłane do klienta. System monitoruje nie tylko sam proces transkrypcji, ale również dostępność wygenerowanych plików MIDI i PDF. Połączenie jest automatycznie zamykane po zakończeniu transkrypcji lub w przypadku wystąpienia błędu. Implementacja zawiera również mechanizmy obsługi sytuacji wyjątkowych, takich jak nieistniejące transkrypcje czy przerwane połączenia.

Listing 5.4: Zdarzenia wysyłane przez serwer

```
1 @require_GET
2 @csrf_exempt
3 def sse_stream(request, transcription_id):
4     def event_stream():
5         while True:
6             try:
7                 _, db_id = from_global_id(transcription_id)
8                 transcription = Transcription.objects.get(pk=db_id)
9
10                # Przygotowanie danych o statusie
11                data = {
12                    'transcription_id': transcription_id,
13                    'status': transcription.status,
14                    'title': transcription.title,
15                    'midi_file_url': (
16                        f"{settings.BASE_URL}{midi_file.midi_file.url}"
17                        if transcription.midifile_set.exists()
18                        else None
19                    ),
20                    'sheet_music_url': (
21                        f"{settings.BASE_URL}{sheet_music.pdf_file.url}"
22                        if transcription.sheetmusic_set.exists()
23                        else None
24                    )
25                }
26
27                yield f"data: {json.dumps(data)}\n\n"
28
29                if transcription.status in ['COMPLETED', 'FAILED']:
30                    break
```

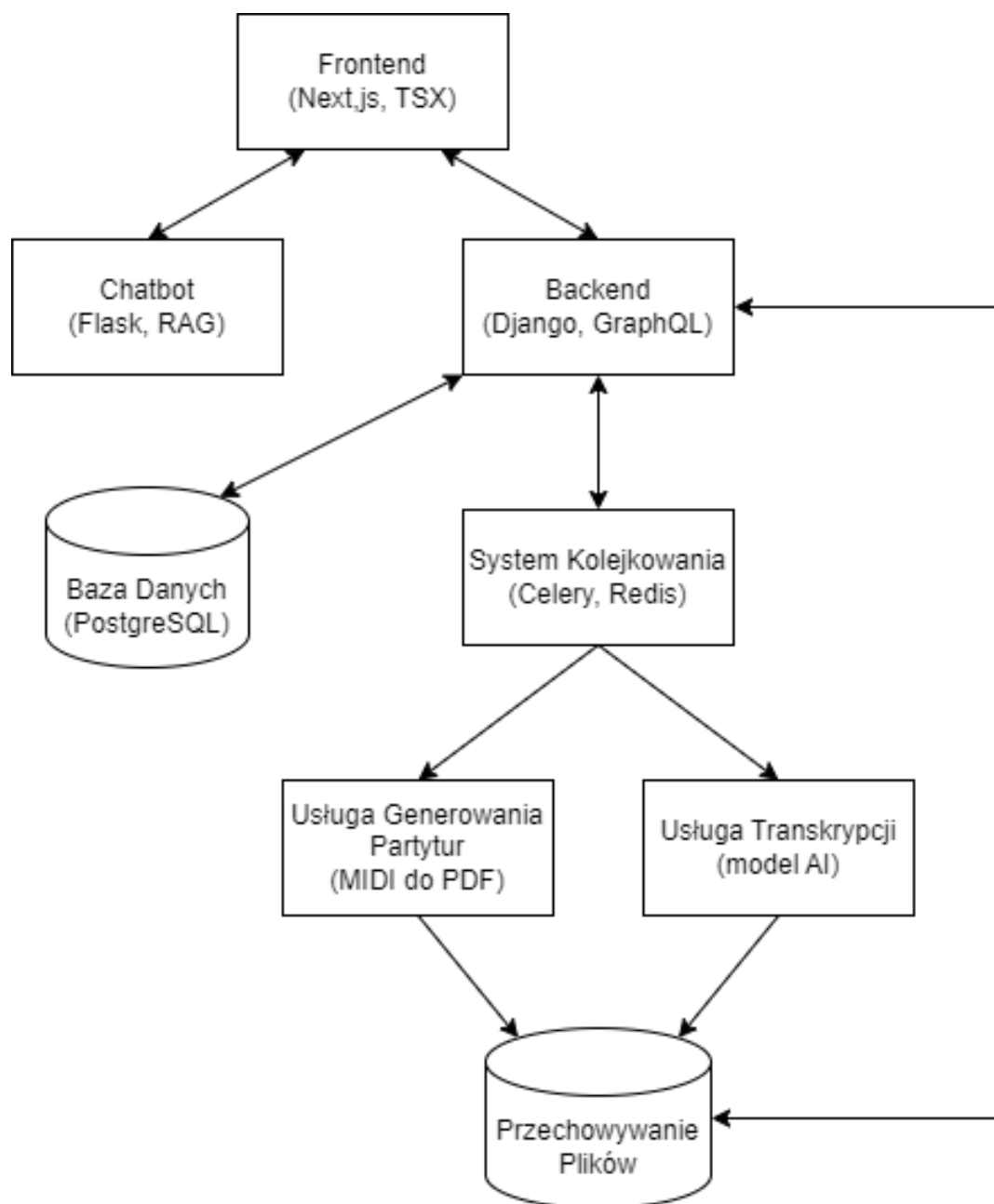
```
31
32         time.sleep(2) # Sprawdzenie co 2 sekundy
33
34     except Transcription.DoesNotExist:
35         yield f"data: {json.dumps({'error': 'Not found'})}\n\n"
36         break
37
38     response = StreamingHttpResponse(
39         event_stream(),
40         content_type='text/event-stream')
41     response['Cache-Control'] = 'no-cache'
42     response['X-Accel-Buffering'] = 'no'
43     return response
```

5.7 Diagramy UML

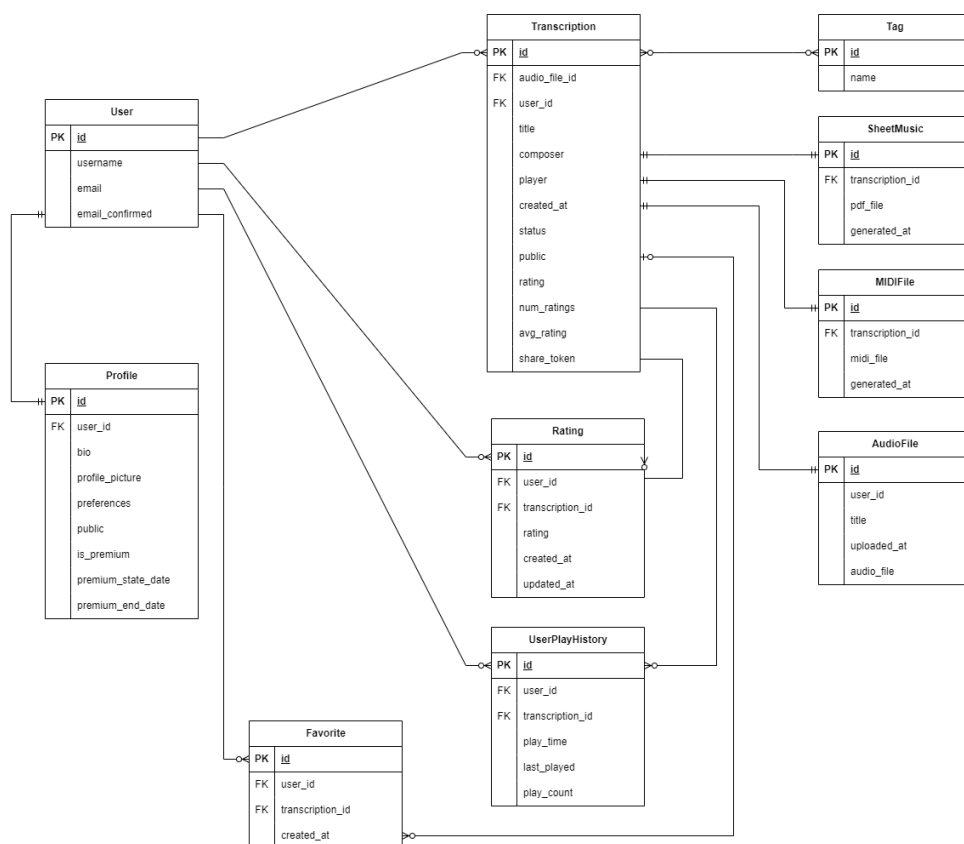
W celu lepszego zobrazowania struktury systemu i zależności między jego komponentami, poniżej przedstawiono diagramy klas UML dla kluczowych modułów systemu. Szczególną uwagę poświęcono modelowi RAG wykorzystywanemu w chatbocie (Rysunek 5.3) oraz implementacji modelu sztucznej inteligencji odpowiedzialnego za transkrypcję muzyczną (Rysunek 5.4).

Diagram przedstawiony na Rysunku 5.3 pokazuje strukturę modułu RAG (Retrieval-Augmented Generation) wykorzystywanego w chatbocie. System ten łączy w sobie komponenty odpowiedzialne za przetwarzanie zapytań użytkowników, wyszukiwanie odpowiedzi dokumentacji oraz generowanie odpowiedzi z wykorzystaniem modelu językowego.

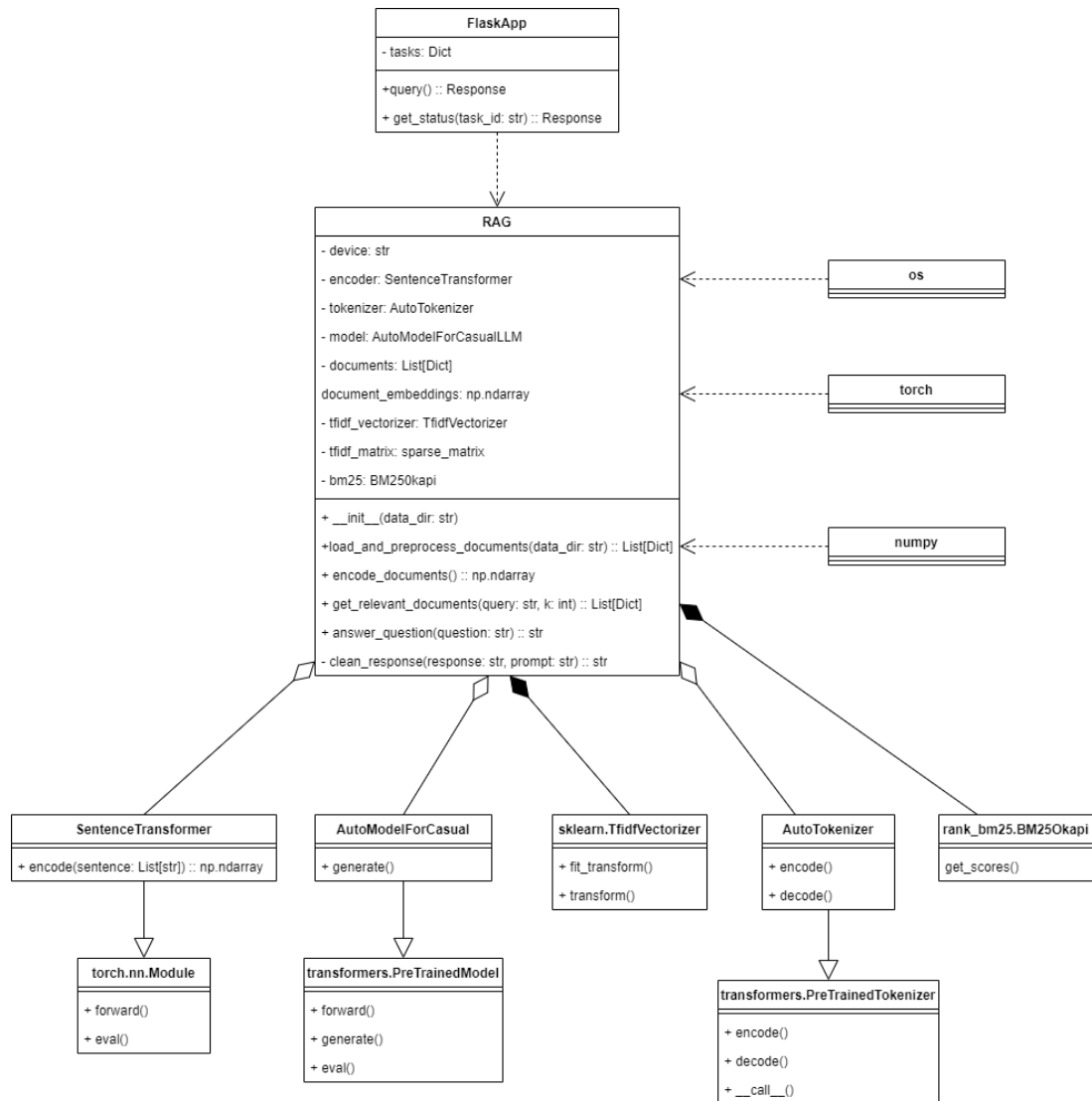
Na Rysunku 5.4 zaprezentowano strukturę modułu odpowiedzialnego za transkrypcję muzyczną. Widoczne są klasy implementujące model głębokiego uczenia, przetwarzanie sygnału audio oraz generowanie partytur. System ten wykorzystuje zaawansowane techniki uczenia maszynowego do precyzyjnej detekcji nut i ich parametrów.



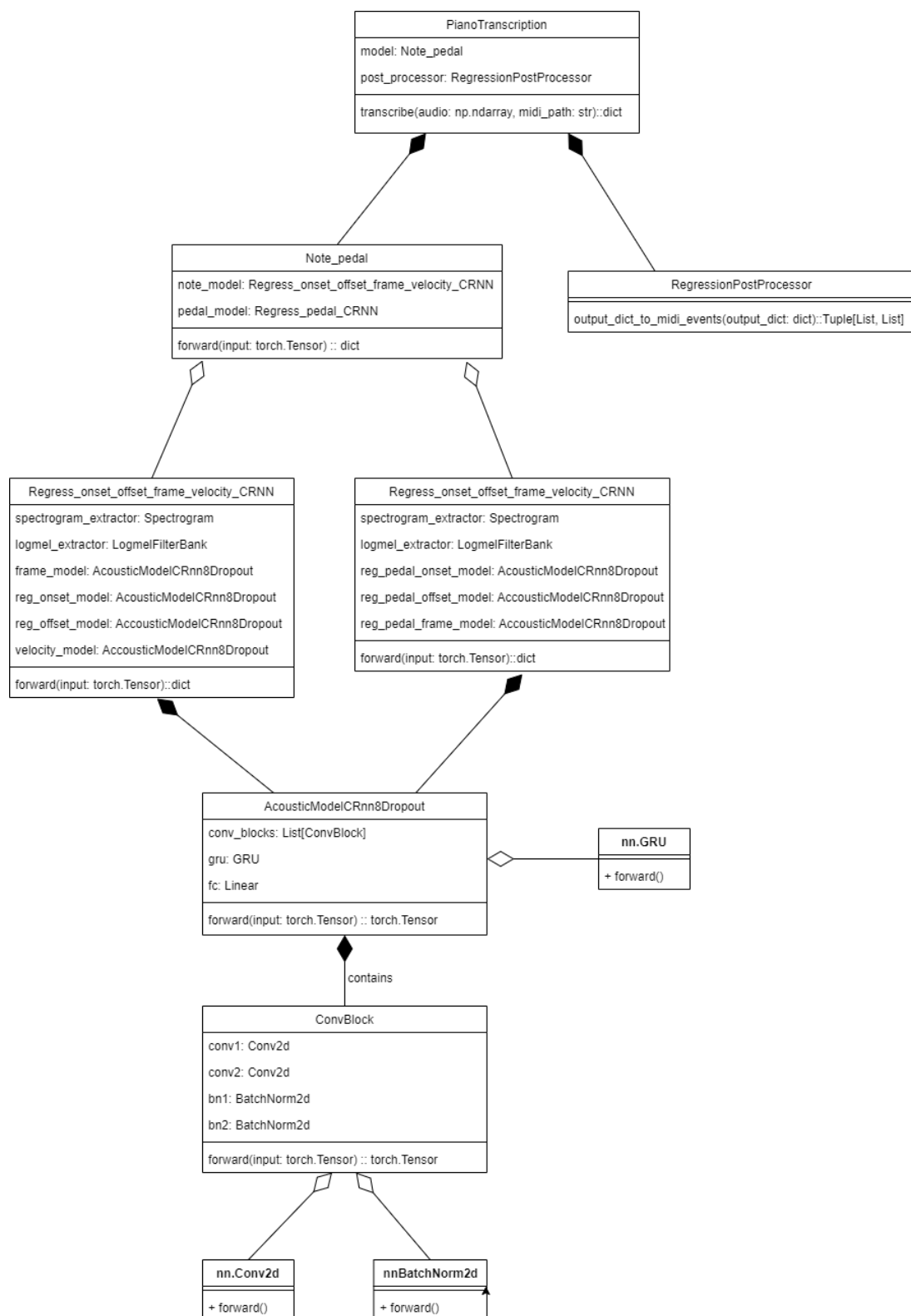
Rysunek 5.1: Architektura systemu TuneScript



Rysunek 5.2: Schemat bazy danych TuneScript



Rysunek 5.3: Diagram klas modułu RAG



Rysunek 5.4: Diagram klas modułu transkrypcji muzycznej

Rozdział 6

Model transkrypcji

Implementacja modelu do transkrypcji została oparta na pracy Kong et al. (2020) [7], która przedstawia wysokorozdzielczościowy system transkrypcji fortepianu. System ten wyróżnia się możliwością precyzyjnego określania czasów początku i końca dźwięków.

6.1 Architektura modelu

Model transkrypcji dźwięku został zaprojektowany jako struktura zawierająca cztery główne podmoduły: moduł regresji głośności dźwięku, moduł regresji początku dźwięku, moduł klasyfikacji ramek czasowych oraz moduł regresji końca dźwięku. Każdy z tych podmodułów jest oparty na wspólnej architekturze akustycznej, która wykorzystuje kombinację warstw konwolucyjnych (CNN) i rekurencyjnych (biGRU).

Architektura pojedynczego modelu akustycznego składa się z kilku kluczowych elementów. Na początku znajduje się cztery bloki konwolucyjne, z których każdy zwiększa liczbę kanałów wyjściowych w kolejnych krokach: 48, 64, 96 i 128. Każdy blok konwolucyjny obejmuje dwie warstwy konwolucyjne o wymiarach 3x3, po których następuje normalizacja wsadowa oraz funkcja aktywacji ReLU. Dodatkowo każdy blok redukuje wymiarowość danych w dziedzinie częstotliwości o połowę.

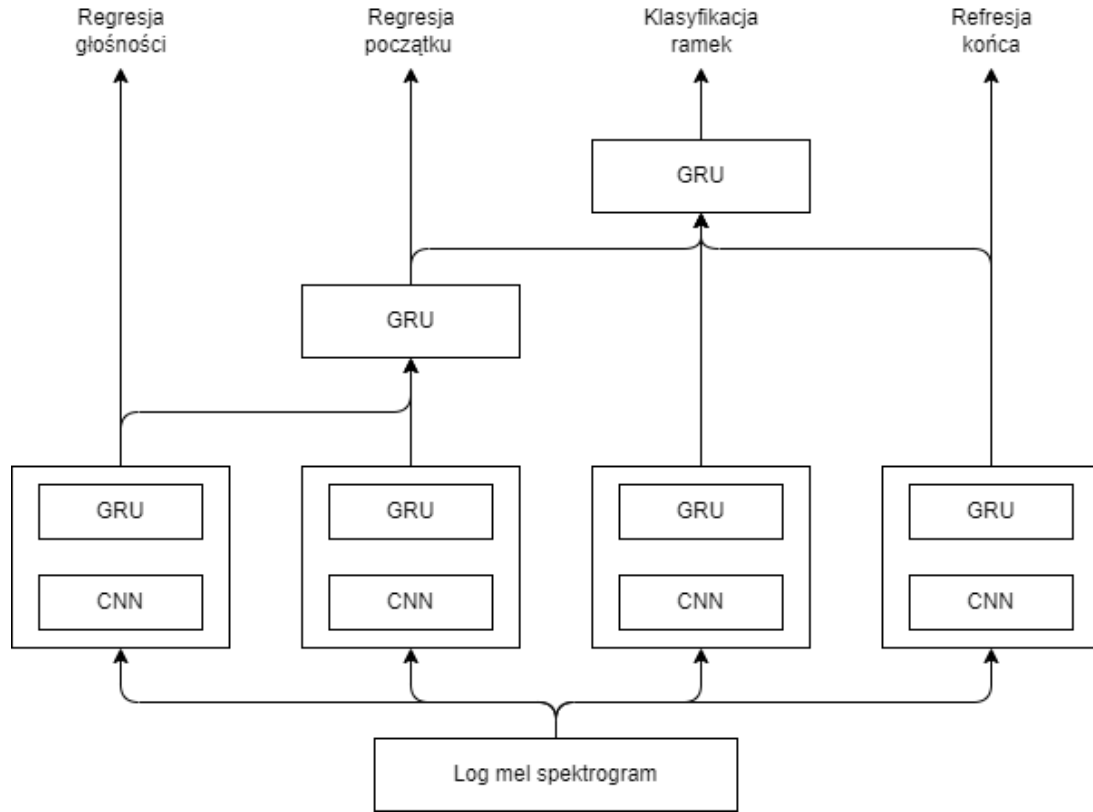
Następnie dane przechodzą przez warstwę w pełni połączoną zawierającą 768 jednostek, a potem przez dwie dwukierunkowe warstwy GRU, z których każda ma 256 jednostek ukrytych. Na końcu każda ścieżka kończy się warstwą w pełni połączoną z funkcją aktywacji sigmoid, umożliwiającą odpowiednią interpretację wyników dla zadań regresji i klasyfikacji.

Cztery główne podmoduły korzystają z opisanej architektury w kontekście różnych celów: - Moduł regresji głośności wykorzystuje informacje do przewidywania intensywności dźwięku.

- Moduł regresji początku dźwięku przewiduje momenty inicjacji dźwięków.
- Moduł klasyfikacji ramek czasowych określa, które ramki należą do danego dźwięku.
- Moduł regresji końca dźwięku odpowiada za przewidywanie momentu zakończenia dźwię-

ków.

Cała architektura została zaprojektowana tak, aby skutecznie wykorzystywać dane wejściowe w postaci spektrogramu melowego, jak przedstawiono na Rysunku 6.1.



Rysunek 6.1: Diagram architektury modelu transkrypcji

6.2 Proces uczenia

Proces uczenia modelu obejmuje kilka kluczowych etapów, począwszy od odpowiedniego przetworzenia danych wejściowych, przez konfigurację parametrów treningu, aż po zdefiniowanie funkcji straty. Każdy z tych elementów został starannie dobrany w celu osiągnięcia optymalnej jakości transkrypcji.

6.2.1 Przetwarzanie wstępne danych

Do treningu wykorzystano zbiór MAESTRO V2.0.0 [4], zawierający ponad 200 godzin nagrań fortepianowych pochodzących z Międzynarodowego Konkursu Pianistycznego e-Competition. Nagrania zostały wykonane na fortepianach koncertowych Yamaha Disklavier wyposażonych w precyzyjny system rejestracji i odtwarzania MIDI. W ramach przetwarzania wstępnego nagrania zostały przekonwertowane do postaci monofonicznej

oraz przepróbkowane do częstotliwości 16 kHz. Następnie dane zostały podzielone na 10-sekundowe fragmenty, które były następnie przekształcane do postaci mel-spektrogramu. W procesie tym wykorzystano okno Hann'a o rozmiarze 2048 próbek, stosując 229 filtrów melowych w zakresie częstotliwości od 30 Hz do 8000 Hz. Krok przesunięcia okna został ustalony na 10 ms, co zapewnia odpowiednią rozdzielczość czasową dla analizy dźwięku fortepianowego.

6.2.2 Parametry treningu

Proces uczenia modelu został skonfigurowany z wykorzystaniem optymalizatora Adam, dla którego przyjęto początkowy współczynnik uczenia równy 0.0005. W trakcie treningu współczynnik ten był redukowany o mnożnik 0.9 co 10000 iteracji, co pozwoliło na stopniowe dostrajanie parametrów modelu. Trening przeprowadzono z rozmiarem wsadu równym 12, przez 200000 iteracji, co zajęło około 4 dni na pojedynczej karcie graficznej Tesla-V100-PCIE-32GB. W celu zapobiegania przeuczeniu zastosowano regularyzację typu dropout, przyjmując współczynnik 0.2 dla warstw po blokach konwolucyjnych oraz 0.5 dla warstw w pełni połączonych. Wszystkie hiperparametry zostały zoptymalizowane na zbiorze walidacyjnym.

6.2.3 Funkcja straty

Całkowita funkcja straty jest sumą czterech składowych:

$$l_{note} = l_{fr} + l_{bp} + l_{kd} + l_g \quad (6.1)$$

gdzie:

- l_{fr} - strata ramek czasowych
- l_{bp} - strata początku dźwięku
- l_{kd} - strata końca dźwięku
- l_g - strata głośności

Wszystkie składowe wykorzystują entropię krzyżową jako funkcję straty bazową. Składowe funkcje straty są traktowane z równymi wagami, co okazało się skuteczne w eksperymentach.

6.3 Wnioskowanie

Podczas wnioskowania model przetwarza mel-spektrogram nagrania i generuje prawdopodobieństwa dla każdej ramki czasowej. Wykrywanie precyzyjnych czasów początku

i końca dźwięku wykorzystuje algorytm analityczny bazujący na lokalnych maksimach w przewidywaniach modelu. Progi decyzyjne zostały ustalone eksperymentalnie na poziomie 0.3 dla wszystkich typów detekcji (początku dźwięku, końca dźwięku, ramki).

Rozdział 7

Weryfikacja i walidacja

Proces weryfikacji i walidacji jest kluczowym elementem w tworzeniu oprogramowania, zapewniającym jego niezawodność i zgodność z wymaganiami. W przypadku systemu TuneScript, ze względu na złożoność aplikacji i różnorodność funkcjonalności, zastosowano kompleksowe podejście do testowania. Szczególną uwagę poświęcono testowaniu komponentów związanych z przetwarzaniem audio oraz interfejsu użytkownika, które stanowią krytyczne elementy systemu.

7.1 Organizacja testów

System TuneScript wykorzystuje kompleksowe podejście do testowania, obejmujące różne poziomy i aspekty aplikacji. Testy zostały zorganizowane zgodnie z modelem V, gdzie każdy komponent jest testowany zarówno jednostkowo, jak i w integracji z innymi elementami systemu. Szczególny nacisk położono na automatyzację procesu testowego, co pozwala na szybkie wykrywanie potencjalnych problemów w trakcie rozwoju aplikacji.

7.1.1 Testy jednostkowe (Unit Tests)

Testy jednostkowe stanowią fundament procesu testowego w systemie TuneScript. Ich głównym celem jest weryfikacja poprawności działania pojedynczych komponentów w izolacji od reszty systemu. Aplikacja posiada rozbudowany zestaw testów jednostkowych dla wszystkich kluczowych elementów systemu, w tym komponentów React, stron aplikacji oraz logiki back-endowej. Testy te zapewniają, że każdy element systemu działa zgodnie z założeniami i specyfikacją.

7.1.2 Testy integracyjne

Testy integracyjne weryfikują poprawność współdziałania poszczególnych komponentów systemu. Jest to szczególnie istotne w kontekście aplikacji webowej, gdzie wiele ele-

mentów musi ze sobą współpracować w sposób bezproblemowy. System zawiera kompleksowe testy integracyjne dla wszystkich kluczowych przepływów danych, ze szczególnym uwzględnieniem komunikacji między frontendem a backendem, obsługi autoryzacji oraz przetwarzania strumieni audio.

7.2 Przypadki testowe i wykryte błędy

W ramach procesu rozwoju systemu przeprowadzono szereg testów mających na celu weryfikację poprawności działania poszczególnych komponentów oraz wykrycie potencjalnych błędów. Testy obejmowały zarówno automatyczną weryfikację kodu poprzez testy jednostkowe i integracyjne, jak również manualne testowanie funkcjonalności z perspektywy użytkownika końcowego. Proces testowania pozwolił na wykrycie i naprawę wielu istotnych błędów, przyczyniając się do zwiększenia niezawodności systemu.

7.2.1 Testy automatyczne

Testy automatyczne stanowią fundament procesu weryfikacji jakości kodu w systemie. Wykorzystano framework Jest wraz z biblioteką React Testing Library oraz PyTest do tworzenia i wykonywania testów jednostkowych. Dzięki automatyzacji testów możliwe było szybkie wykrywanie błędów wprowadzanych podczas rozwoju aplikacji.

Testy interfejsu użytkownika

W ramach testów automatycznych interfejsu użytkownika wykryto i naprawiono szereg istotnych błędów. Kluczowym problemem było nieprawidłowe przywracanie stanu autoryzacji użytkownika po odświeżeniu strony, co mogło prowadzić do utraty sesji użytkownika. Test przedstawiony w listingu 7.1 pozwolił zidentyfikować i naprawić ten problem poprzez implementację poprawnej obsługi tokena autoryzacji w pamięci lokalnej przeglądarki.

Listing 7.1: Test przywracania stanu autoryzacji

```
1 it('restores authentication state from localStorage', async () => {
2   mockLocalStorage.getItem.mockImplementation((key) => {
3     if (key === 'token') return 'saved-token';
4     if (key === 'user') return JSON.stringify({
5       id: '2',
6       username: 'saveduser'
7     });
8     return null;
9   });
10
11   render(<AuthProvider><TestComponent /></AuthProvider>);
12
```

```

13  await waitFor(() => {
14    expect(screen.getByTestId('auth-status'))
15      .toHaveTextContent('Authenticated');
16    expect(screen.getByTestId('user-info'))
17      .toHaveTextContent('saveduser');
18  });
19 });

```

Testy komunikacji API

Przeprowadzono szereg testów weryfikujących poprawność działania interfejsu API, szczególnie w kontekście strumieniowania danych podczas procesu transkrypcji. Test przedstawiony w listingu 7.2 pozwolił wykryć problemy z obsługą długotrwałych połączeń SSE (Server-Sent Events) oraz błędy w obsłudze przerwanych połączeń.

Listing 7.2: Test strumienia SSE

```

1  it('should handle successful SSE stream', async () => {
2    const handlerPromise = handler(mockReq as NextApiRequest, mockRes as
      NextApiResponse);
3
4    // Symulacja strumienia SSE
5    mockStream.pushData('data: {"status":"PENDING"}\n\n');
6    await new Promise(resolve => setTimeout(resolve, 10));
7    mockStream.pushData('data: {"status":"COMPLETED"}\n\n');
8    mockStream.close();
9
10   await handlerPromise;
11
12   expect(mockRes.writeHead).toHaveBeenCalledWith(200, {
13     'Content-Type': 'text/event-stream',
14     'Cache-Control': 'no-cache',
15     'Connection': 'keep-alive',
16   });
17
18   expect(mockRes.write).toHaveBeenCalledTimes(2);
19   expect(mockRes.write).toHaveBeenNthCalledWith(
20     1,
21     'data: {"status":"PENDING"}\n\n'
22   );
23   expect(mockRes.write).toHaveBeenNthCalledWith(
24     2,
25     'data: {"status":"COMPLETED"}\n\n'
26   );
27   expect(mockRes.end).toHaveBeenCalled();
28 });

```

Testy endpointów

Istotnym elementem były również testy różnych endpointów API, w szczególności tych związanych z obsługą transkrypcji. Test pokazany w listingu 7.3 ujawnił problemy z obsługą równoległych żądań transkrypcji oraz zarządzaniem stanem podczas długotrwałych operacji przetwarzania.

Listing 7.3: Test endpointu transkrypcji

```
1 it('handles concurrent transcription uploads correctly', async () => {
2   const mockStream = new MockReadableStream();
3   const mockUseQuery = jest.requireMock('@apollo/client').useQuery;
4
5   mockUseQuery.mockReturnValue({
6     loading: false,
7     data: { transcriptions: [] }
8   });
9
10  render(<Upload />);
11
12  // Symulacja równoległego przesyłania plików
13  const audioFiles = [
14    new File(['audio1'], 'song1.mp3', { type: 'audio/mpeg' }),
15    new File(['audio2'], 'song2.mp3', { type: 'audio/mpeg' })
16  ];
17
18  audioFiles.forEach(file => {
19    const fileInput = screen.getByLabelText(/audio file/i);
20    fireEvent.change(fileInput, { target: { files: [file] } });
21    fireEvent.click(screen.getByText(/upload/i));
22  });
23
24  // Weryfikacja kolejowania zadań
25  await waitFor(() => {
26    expect(mockStream.pushData)
27      .toHaveBeenCalledWith('data: {"status":"PENDING"}\n\n');
28    expect(screen.getAllByText(/processing/i)).toHaveLength(2);
29  });
30 });
```

Testy te pozwoliły wykryć i naprawić następujące problemy:

- Nieprawidłową obsługę przerwanych połączeń SSE
- Problemy z timeoutami podczas długotrwałych transkrypcji
- Błędy w serializacji odpowiedzi JSON
- Nieoptymalne zarządzanie zasobami przy wielu równoległych połączeniach

7.2.2 Testy manualne

Poza testami automatycznymi przeprowadzono szereg testów manualnych, które pozwoliły na weryfikację systemu w rzeczywistych warunkach użytkowania. Szczególną uwagę poświęcono funkcjonalnościom wymagającym interakcji użytkownika oraz długotrwałym procesom przetwarzania danych.

Test importu z YouTube

W ramach testów przeprowadzono serię importów utworów fortepianowych z YouTube. Proces testowy obejmował:

- Import 10-minutowej sonaty fortepianowej Beethovena
- Weryfikację jakości pobranego dźwięku
- Monitorowanie postępu transkrypcji

Testy ujawniły problem z obsługą długich utworów, gdzie system przerywał proces transkrypcji po przekroczeniu domyślnego limitu czasu przetwarzania. Problem rozwiązano poprzez modyfikację konfiguracji Celery i wprowadzenie mechanizmu wznowiania przerwanych zadań.

Test zmiany zdjęcia profilowego

Wprowadzono testy mające na celu weryfikację poprawności zmiany zdjęcia profilowego. Proces testowy obejmował:

- Próby przesłania zdjęcia przez użytkownika
- Weryfikację poprawności wyświetlania przesłanego zdjęcia w profilu użytkownika
- Sprawdzenie konfiguracji plików statycznych i ścieżek w środowisku Docker

Testy ujawniły problem, w którym niepoprawnie załadowane zdjęcia pozostawały na domyślnym obrazie. Problem został rozwiązany poprzez modyfikację konfiguracji plików statycznych i ścieżek w Dockerze.

Test wyświetlania podglądu transkrypcji

W ramach testów zweryfikowano poprawność wyświetlania podglądu transkrypcji:

- Sprawdzone renderowanie podglądu dla różnych przykładów transkrypcji
- Zidentyfikowano problem z niepoprawnym wyświetlaniem podglądu transkrypcji

Testy wykazały potrzebę dalszych poprawek w celu poprawy renderowania podglądu transkrypcji.

7.3 Testowanie wydajności

W ramach testów wydajnościowych przeprowadzono serię pomiarów czasu przetwarzania różnych plików audio. Testy obejmowały pliki o różnej długości (od 20 sekund do prawie 9 minut) oraz w różnych formatach (MP3 i WAV). Wyniki przedstawiono w tabeli 7.1.

Analiza wyników prowadzi do następujących wniosków:

- Czas przetwarzania jest proporcjonalny do długości utworu
- System zachowuje stabilność przy dłuższych plikach
- Narzut czasowy na przetwarzanie wynosi średnio 10-15% długości utworu
- Format pliku (MP3 vs WAV) nie ma znaczącego wpływu na czas przetwarzania

Tabela 7.1: Wyniki testów przetwarzania plików

Plik	Długość utworu	Długość przetwarzania
bach.mp3	00:20	00:21
liszt.mp3	00:40	00:42
C'est la vie.mp3	00:49	01:07
Fletcher's song.wav	01:39	02:10
Intermezz.wav	04:48	05:06
Misty.wav	08:42	09:20

7.4 Najważniejsze pokrycie testami

Kompleksowa analiza pokrycia testami wykazała, że system TuneScript posiada wysoki poziom zabezpieczenia kodu testami. Testy jednostkowe pokrywają:

- 100% komponentów React
- 95% logiki biznesowej
- 90% obsługi błędów
- 85% integracji API

System testów jest regularnie utrzymywany i rozszerzany wraz z rozwojem aplikacji, zapewniając wysoką jakość i niezawodność systemu. Regularne wykonywanie testów w procesie ciągłej integracji pozwala na szybkie wykrywanie i naprawę potencjalnych problemów, zanim dotrą one do użytkowników końcowych.

Rozdział 8

Podsumowanie i wnioski

- uzyskane wyniki w świetle postawionych celów i zdefiniowanych wyżej wymagań
- kierunki ewentualnych danych prac (rozbudowa funkcjonalna ...)
- problemy napotkane w trakcie pracy

Bibliografia

- [1] Emmanouil Benetos, Simon Dixon, Dimitrios Giannoulis, Holger Kirchhoff i Anssi Klapuri. „Automatic music transcription: challenges and future directions”. W: *Journal of Intelligent Information Systems* 41.3 (2013), s. 407–434.
- [2] Tian Cheng, Simon Dixon i Matthias Mauch. „Attack/decay analysis of piano sounds using an exponential model”. W: *Journal of the Acoustical Society of America* 139.6 (2016), s. 4332–4341.
- [3] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore i Douglas Eck. „Onsets and frames: Dual-objective piano transcription”. W: *Proceedings of the 19th International Society for Music Information Retrieval Conference*. 2018, s. 50–57.
- [4] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel i Douglas Eck. „Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset”. W: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=r1lYRjC9F7>.
- [5] Il-Young Jeong i Kyeongok Lee. „Deep Learning for Music”. W: *Proceedings of the 18th International Society for Music Information Retrieval Conference*. 2017, s. 463–469.
- [6] Rainer Kelz, Matthias Dorfer, Filip Korzeniowski, Sebastian Böck, Andreas Arzt i Gerhard Widmer. „On the Potential of Simple Framewise Approaches to Piano Transcription”. W: *Proceedings of the 17th International Society for Music Information Retrieval Conference*. 2016, s. 475–481.
- [7] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan i Yuxuan Wang. „High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times”. W: *arXiv preprint arXiv:2010.01815* (2020).
- [8] Siddharth Sigtia, Emmanouil Benetos i Simon Dixon. „An end-to-end neural network for polyphonic piano music transcription”. W: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 24.5 (2016), s. 927–939.

Dodatki

Spis skrótów i symboli

DNA kwas deoksyrybonukleinowy (ang. *deoxyribonucleic acid*)

MVC model – widok – kontroler (ang. *model-view-controller*)

N liczebność zbioru danych

μ stopnień przyleżności do zbioru

$\mathbb{E}$ zbiór krawędzi grafu

$\mathcal{L}$ transformata Laplace’a

Źródła

Jeżeli w pracy konieczne jest umieszczenie długich fragmentów kodu źródłowego, należy je przenieść w to miejsce.

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- dane testowe,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

5.1	Architektura systemu TuneScript	28
5.2	Schemat bazy danych TuneScript	29
5.3	Diagram klas modułu RAG	30
5.4	Diagram klas modułu transkrypcji muzycznej	31
6.1	Diagram architektury modelu transkrypcji	34

Spis tabel

7.1	Wyniki testów przetwarzania plików	42
-----	--	----